rsna-knee-abnormality-detection

1st place solution — Better labels, anatomy-aware MRI models

Rank	1
Team	Team Kangaroo
Author	sakaku
Topic ID	752474
Version	Original

Source: https://www.kaggle.com/competitions/rsna-knee-abnormality-detection/discussion/752474
Retrieved with Kaggle CLI on 2026-10-27.

Thank you to RSNA, the radiologists, and Kaggle for organizing this competition, and to my teammate @tascj0 for a collaboration I learned so much from.

This post covers my three main contributions to our team's solution: **high-quality Astra labels, anatomical BBox annotations, and the models I contributed to the final ensemble**. For my teammate's work, please see @tascj0's write-up.

The training set contained **4,407 MRI studies with radiology reports: 58 had human target labels, and 4,349 needed labels derived from the available evidence**. The reports contained much richer information than a twelve-element label vector. The challenge was turning that information into supervision an image model could learn from: a finding might be uncertain, belong to a different anatomical compartment, or fall short of the competition's definition.

High-quality Astra labels. My GPT-6 Astra annotation pipeline investigated each MRI study, requested additional visual evidence, and produced continuous probabilities for all twelve findings. These labels and the probability-assignment protocol became shared team assets. In a matched CoAtNet (224 px) comparison, replacing our earlier multimodal labels with Astra labels raised public AUC from **0.922 to 0.935**.

Anatomical BBox annotations. I developed the anatomical annotation and detector pipeline that supplied slice-indexed regions for spatial supervision. These boxes guided the classifier toward relevant anatomy while preserving global context. My teammate separately supplied lesion annotations, which we also incorporated into my models. Together, study labels and spatial annotations addressed both what the model should predict and where it should look.

Contribution to the final ensemble. I trained a complementary three-model group: two MaxViT v2 Base models with different label recipes and one CoAtNet-rMLP-2 model. Their equal-weight blend supplied **50% of the final team prediction**, with my teammate's model group supplying the other half. The combined submission achieved **0.969 public / 0.960 private AUC** and first place. This post follows those contributions from annotation to model training and final integration.

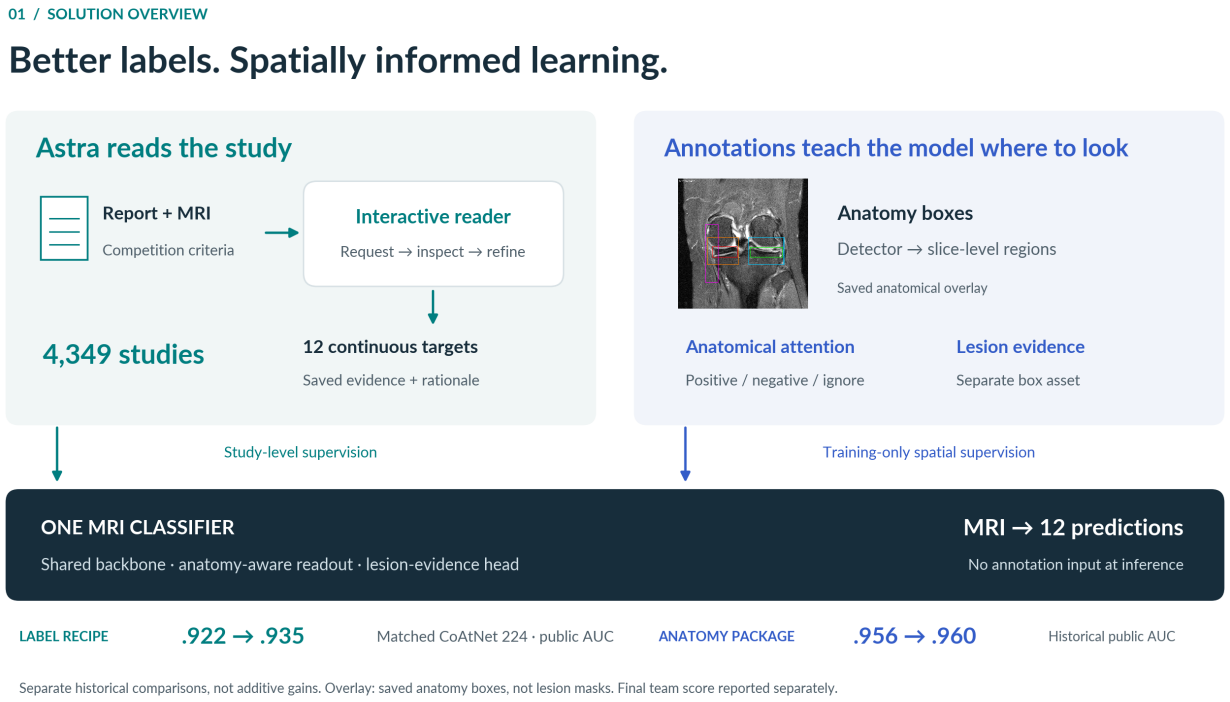


Figure 1. Two annotation paths feed the same MRI classifier: continuous study-level targets and training-only spatial supervision. Anatomy and lesion boxes are separate assets. The two score changes summarize separate historical comparisons, not additive gains or final team performance.

Final team inference notebook

1. High-quality labels with GPT Astra

Our annotation pipeline developed in two steps: **single-pass labeling with Hulu and Qwen, followed by agentic labeling with GPT Astra**. The earlier work helped us refine the prompts and rules for judging each finding. Astra built on those lessons and added the ability to request more images and check its assessment over multiple steps. The image classifier learned from the resulting labels; it needed neither reports nor an annotation agent at inference.

First, learn how to judge the findings

Before Astra, we used **ClinFusion, Hulu and Qwen** to read the report alongside a fixed packet of **21 MRI slices: seven sagittal, seven coronal and seven axial**. Hulu and Qwen produced each target score in a single inference request using the supplied material. They could reason over those inputs, but could not call a viewer to obtain additional evidence.

Reader	How it produced labels	What we retained
ClinFusion-32B	Separate YES/NO token scores for each target	An early baseline and structured diagnostic explanations
Hulu-Med-Flash-Preview-27B	Constrained yes/no token scoring	A probability source, including a dedicated Synovitis recipe
Qwen3.8-Flash-Next-FP8	Per-target reasoning followed by a continuous number	A complementary probability source

Hulu's dedicated Synovitis pass kept the same slices at higher native detail and removed selected synovial wording from the report to reduce anchoring. Qwen was self-hosted and made twelve requests per study. ClinFusion's explanations helped us inspect whether a claim came from the report or the images; its probabilities were not used in the final blend. Hulu scored targets from yes/no token probabilities with thinking disabled, while Qwen generated a continuous number after reasoning. Probability scoring and diagnostic-text extraction were separate Hulu calls.

Developing these readers taught us how to frame the task: follow the competition's definitions, distinguish report statements from visible image evidence, avoid treating an unmentioned finding as negative, and preserve uncertainty in the score. **We carried these lessons into Astra's prompt design.** Its prompts were informed by the earlier work rather than designed from scratch.

The next step was to let the reader decide **what else it needed to see**. Astra could request slices, inspect a region in another sequence, and revisit its assessment before producing the final probabilities. This made the annotation process agentic. Hulu and Qwen still contributed complementary labels to the final mixture.

Define the target before asking for a probability

Astra estimated whether each **competition target criterion** was met. This required more than extracting an abnormality from the report. Mild ACL signal changes did not establish a high-grade tear; a small amount of joint fluid did not establish the moderate-or-large Effusion target. Medial, lateral and patellofemoral findings also had to stay attached to the correct compartment.

The report was useful context, but silence was not an explicit negative and uncertain wording was not a definite positive. We asked the reader to reconcile the report with the available images and retain uncertainty when the required evidence was missing. This target-specific interpretation was shared by the probability sources.

Then, let Astra gather its own evidence

A fixed contact sheet could leave a specific finding unresolved. With Astra, that uncertainty could trigger another look: a neighboring slice, a closer view, or a complementary sequence.

The production pipeline used **GPT-6 Astra**, with one conversation per study. Each study ran unattended; the interaction was between Astra and its viewing tools. It received the report, the twelve target definitions, and sequence overviews. Rather than fixing every detailed view upfront, we gave the reader a bounded image-viewing interface.

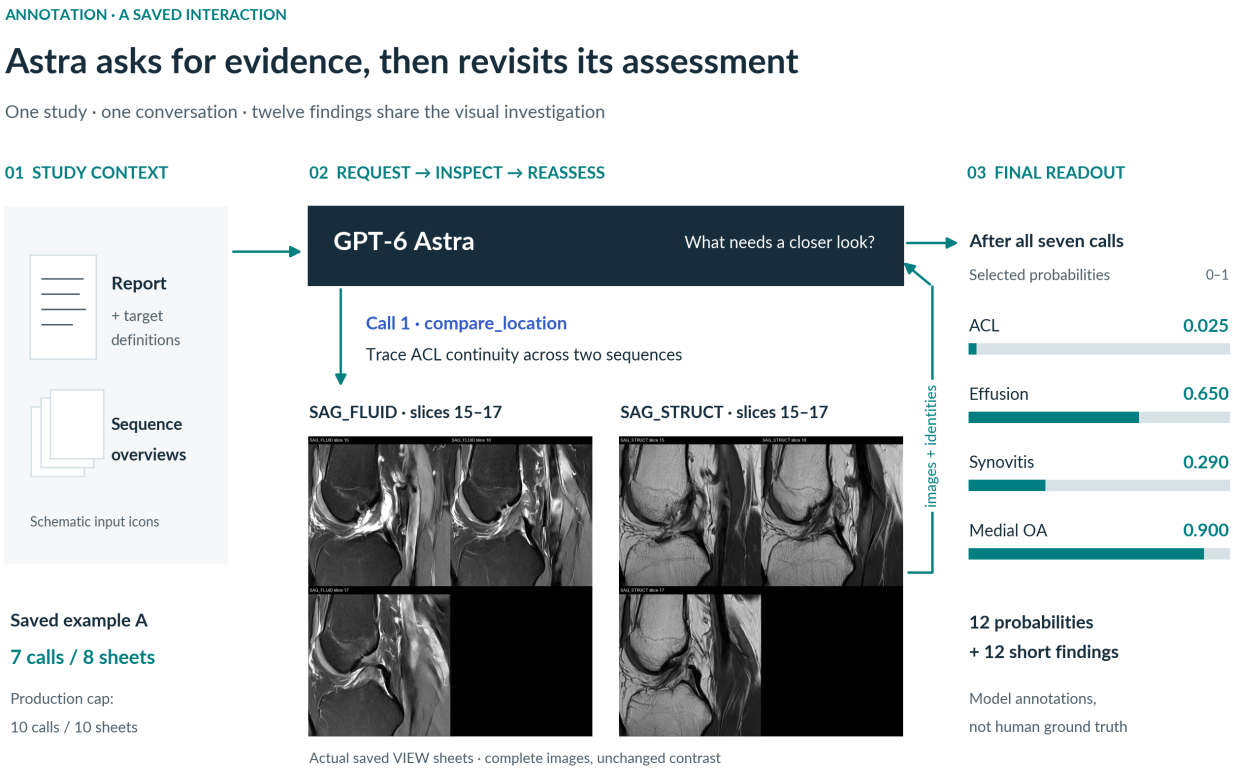


Figure 2. An actual Astra session connects a targeted request, returned MRI views and continuous outputs. The two image sheets come from its first viewer call; probabilities are final outputs after all seven calls and eight sheets. They are model annotations, not expert ground truth. The production limit was ten calls and ten sheets per study.

Two viewer operations covered the core interactions:

Operation	What the annotator could do
`view_slices`	Select slices, crop a region, and adjust the display window with the original view alongside it
`compare_location`	Request neighboring slices around a candidate location in complementary sequences

The viewer returned images and their series/slice identities. Cross-sequence locations were candidates derived from DICOM geometry, so the reader still had to check the anatomy. Astra v1 had a budget of **ten tool calls and ten detailed image sheets** for the whole study. At the end it returned twelve probabilities and twelve short findings.

A saved annotation session

Figure 2 follows a saved production session. The first request compared sagittal fluid-sensitive and structural views around slice 16, asking the model to inspect ACL continuity. It retrieved slices 15–17 in both sequences. Later calls inspected coronal and axial views, the menisci, cartilage, and fluid-related findings. The session used seven calls and eight detailed sheets in total.

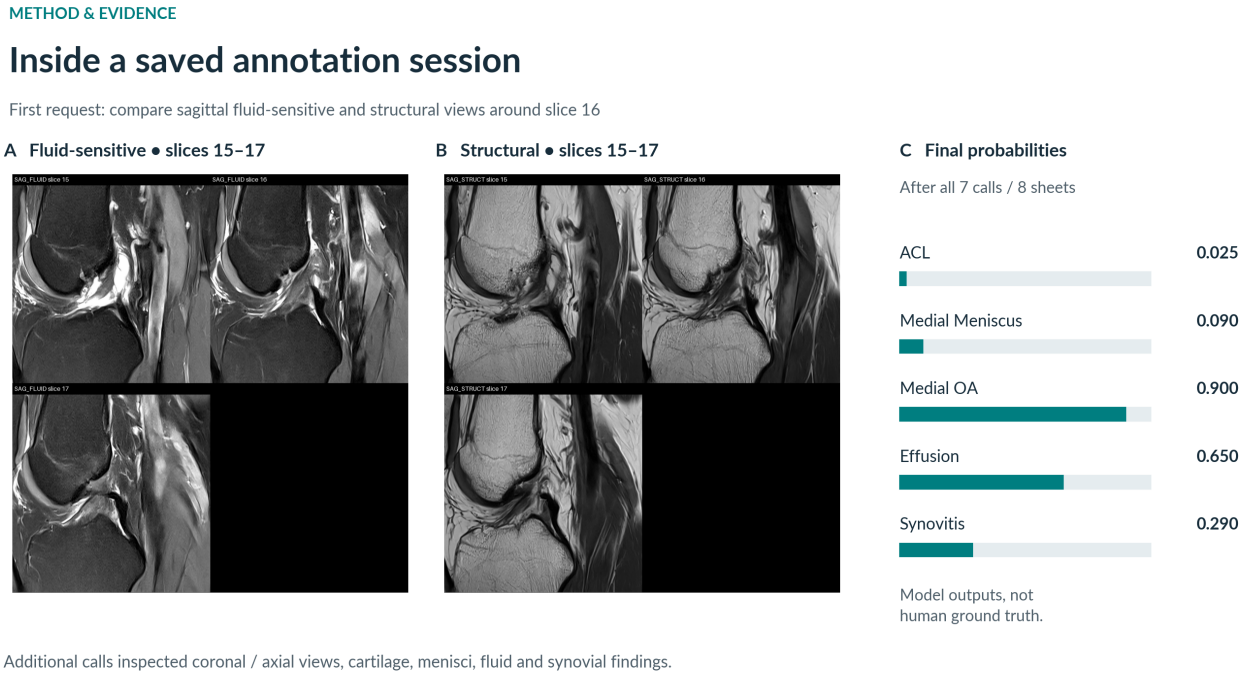


Figure 3. The first viewer call returned these two complementary MRI sheets. The probabilities are selected final outputs after the complete seven-call session, not expert ground truth.

The final output illustrates why I kept probabilities. The annotator assigned **0.025 to ACL**, **0.65 to Effusion**, and **0.29 to Synovitis**. Its text described uncertainty around the effusion threshold and whether synovial thickening was established. The student received those differences instead of three forced yes/no decisions.

The annotator controls both **which evidence to inspect** and **how much certainty to express**. This example shows the interaction; the student comparison below measures the value of the resulting label pipeline as a whole.

Continuous probabilities

We retained the reports as privileged training information, with fixed preprocessing for each annotation source. In the Astra pipeline, we consistently removed selected synovial clauses to reduce anchoring on report wording. The short diagnostic text was useful for inspecting the labels, but the image classifier trained on the probabilities.

An earlier readout experiment reinforced this choice. On the same acquired evidence, six-bin outputs gave **0.8832** development AUC, compared with **0.9363** for continuous outputs; a fresh repeat of the complete continuous method gave **0.9392**. I therefore kept the continuous formulation and checked probability quality as well as ranking during annotation development.

From an annotation session to a training dataset

One conversation per study reused context across all twelve targets. We kept the output to probabilities and concise findings, and measured cached input, uncached input and output tokens separately. Anatomical and lesion coordinates were produced through separate tasks. Each accepted case was saved with its source identities, so interrupted jobs resumed incomplete cases without regenerating finished labels.

The released source covered all **4,349 studies × 12 targets = 52,188 probability–text pairs**. Release checks required unique study–target keys, finite probabilities, complete coverage and valid source-image identities. Raw responses and viewed images stayed linked to each record, making disagreements traceable to the evidence the annotator actually saw.

In my original Astra production run, conversations were isolated to the current case's report, images and MRI-viewing tools, with project history, shell and web access disabled. An independent source audit found no direct injection of the 58 development cases' answers or priors into that path. This audit did not cover later teammate-provided sources. Earlier prompt and report-processing choices had used development feedback, so the annotation scores remain development results.

Comparing the annotation sources

On the same 58 human-labelled development studies, the retained source versions gave:

Annotation source	Macro AUC
ClinFusion	0.8737
Hulu, including specialized Synovitis	0.9093
Qwen Next	0.8955
Astra v1	0.9364
Astra v2	0.9383

Astra v2 changed the initial image presentation and reasoning budget; its small development increase was not statistically confirmed. These report-assisted annotation scores describe individual sources. We judged their usefulness for training through downstream model comparisons and complementary predictions.

Downstream evidence

A matched **CoAtNet (224 px)** comparison illustrates the value of this work. The control already used multimodal labels: Hulu, including its specialized Synovitis source, with a GPT-5.6 Sol contribution for Contusion. Replacing that supervision with Astra labels produced the main gain below.



Figure 4. Historical matched student runs, before the final models. The main gain is from stronger supervision; the additional 0.001 from the early blend is not a confirmed advantage. This is not a cumulative ablation of the winning ensemble.

Final label recipes

For my final models, I used two label recipes. The main recipe, **Mix5**, averaged raw probabilities with the same weights for every target:

Source	Weight	Source variation
Astra v1	30%	Medium reasoning; overview/contact-sheet presentation
Astra v2	20%	Low reasoning; individual-image initial presentation
Teammate Astra	30%	Separately produced annotations
Hulu	10%	Additional image/report source
Qwen Next	10%	Additional image/report source

The second recipe used **probability-calibrated focus-guided Astra labels shared by @tascj0**. One MaxViT v2 Base model trained on this source alone, and it worked particularly well: its development-trained version achieved **0.955 private AUC**, the highest among the five single models compared below. We retained the full-data version in the final ensemble, adding supervision diversity alongside backbone diversity. For details of how my teammate produced these labels, please see his write-up. ClinFusion probability labels and diagnostic text were not inputs to these final three models.

The blend used one formula for every study and target. An earlier fusion audit found that the apparent advantage of flexible target-specific weights disappeared under leave-one-out evaluation. That encouraged a global mixture with fewer adjustable choices; it did not establish the final weights as uniquely optimal.

2. Location supervision: from BBoxes to training signals

Study labels tell the model whether a finding is present, but do not identify the relevant part of the image. We built **two separate spatial datasets**: anatomical regions for six location-dependent targets, and lesion-level evidence for all twelve targets. A normal meniscus still has an anatomical region; a lesion box has a different meaning.

The motivation came partly from an earlier compartment-restricted model: giving the meniscus heads the corresponding half-image raised lateral-meniscus development AUC from **0.8236 to 0.8820** and public AUC from **0.936 to 0.938**, but required about **2.2× as many backbone image evaluations**. The small development set left the gain uncertain. It nevertheless made spatial evidence worth pursuing. In the final design, we used annotations to guide readouts on a shared feature map, retaining global context without another crop-backbone pass.

We also checked localization separately from diagnostic text. An early coordinate reader often retained similar report-driven outputs after its images were replaced. We therefore required the anatomical annotation path to be checked against image content rather than accepting coordinates because the accompanying diagnosis sounded plausible.

Anatomical BBox annotation

The anatomical annotation task used **GPT-6.1 Sol with low reasoning effort**, reading one 448-pixel slice at a time. The prompts were specific to the view and laterality and described which structures to include. They asked for the visible anatomical region where a condition is assessed, without deciding whether that condition was present. Invisible structures were omitted.

The output was a JSON list containing a class name and `bbox_2d = [x1, y1, x2, y2]`, with integer coordinates in `[0,1000]`, a top-left origin, x increasing rightward and y downward. We retained the exact source image and its transform. Normalized coordinates were mapped through the actual rendered image into the canonical physical canvas; they were not treated as pixel coordinates directly.

The raw prompt covered eight candidate classes. The downstream detector used six: **MCL, medial meniscus, lateral meniscus, medial OA, lateral OA and patellofemoral OA**. ACL and Baker's region boxes were not included in this detector. The consumed annotation snapshot contained **39,415 images from 3,823 studies**. These were machine-generated coarse regions, not hand-drawn segmentation masks.

Expand to all slices with a detector

Annotating a few selected slice positions was not enough for a student using depth jitter and variable window sampling. It needed spatial supervision at the slice identities actually selected during training.

We trained a **DINOv3 ConvNeXt-B backbone with an adapter and a six-layer DETR decoder**, starting from pretrained detector weights. Two study-disjoint folds provided out-of-fold boxes for the 4,349 report-labelled studies: each detector predicted the studies excluded from its own training. This is detector OOF generation, distinct from the classifier's development split.

The detector used a 512-pixel, 140 mm canvas, batch size 16, a peak learning rate of 10^{-4} and cosine decay. The full-data annotation expansion allowed up to sixteen epochs and selected EMA checkpoints by held-out mAP50. Training used modest scale, translation and intensity augmentation; flips, Mosaic and MixUp were disabled to preserve the anatomical convention.

The selected folds reached **0.952 and 0.944 mAP50 against the annotation reference**. We also checked center agreement, image substitution, laterality, background predictions and coverage by class and sequence role. Detection agreement with an AI annotator was not treated as expert anatomical ground truth.



Figure 5. The detector expands sparse GPT annotations to a slice-indexed training asset. Supervision is enabled only for accepted target–role combinations; a blank cell means ignore, not a negative label.

Positive, reliable negative, or ignore

The exported dataset covered **673,215 slices**. Each slice-target entry had one of three states:

State	Meaning for the training loss
Positive region	A retained anatomical box supplies spatial support
Reliable negative	The target structure is judged absent from this view; suppress unsupported attention
Ignore	No spatial loss for this entry

We accepted **17 target–role combinations**. For example, medial-meniscus supervision was used in the coronal fluid-sensitive and coronal structural roles; lateral-meniscus supervision additionally used sagittal fluid-sensitive views. Axial meniscus and tibiofemoral OA combinations were excluded. An empty detector output became a reliable negative only under separately checked absence rules. Otherwise it stayed unknown.

This distinction matters in practice. Calling every empty output negative would teach the student the detector's misses; ignoring all empty outputs would remove supervision against attention on views where the

structure is absent. We therefore stored the state explicitly rather than reconstructing it later from the presence of a box.

The 512-, 384- and 256-pixel exports share the same physical canvas. Boxes are transformed together with the student images during CPU and GPU augmentation, and the lookup is keyed to the original series and slice identity.

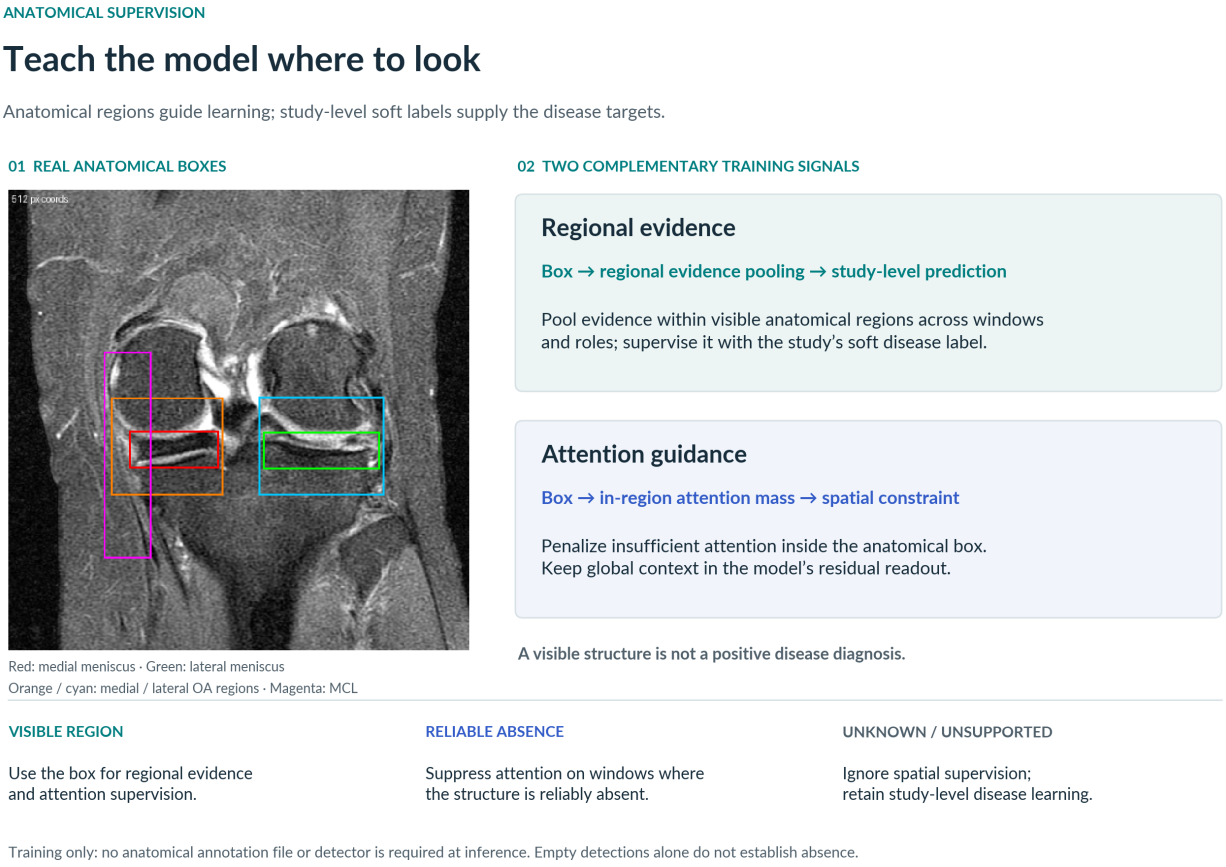


Figure 6. How anatomical boxes become training signals. Left: a real saved coronal overlay. Right: regional evidence pooling and attention guidance. A visible structure does not imply disease; reliable absence constrains window attention, while unknown entries receive no spatial supervision. These are anatomical regions, not lesion segmentations.

Lesion BBoxes: direct evidence for all twelve targets

My teammate supplied a separate **GPT-6.1 Sol high** lesion-annotation archive covering **1,800 studies, 3,616 lesions and 111,029 per-slice boxes**. The agent annotated selected findings on individual slices of the chosen fluid-sensitive series. The records contain the series UID, SOP instance UID, finding identity, normalized source-image coordinates and visibility information.

The distinction between not annotated and absent is especially important here. Only requested findings were covered. A study outside the selected cohort, an unrequested finding or a `not_visible` entry was not a negative example.

For the lesion loss, we required an eligible report-labelled training study, a requested finding, matching series/slice geometry and a fixed Astra-based case-positive eligibility score of at least 0.5. This eligibility source was held fixed when changing the main training-label recipe, including the focus-guided model. Only the **center**

slice of the three-slice window supplied its lesion boxes, and all three slices had to be valid, distinct and undropped. Multiple boxes formed a geometric union for the MIL pooling region. An explicitly negative requested finding could supervise absence within an otherwise eligible case; missing boxes were ignored.

The lesion index was rebuilt for the exact training complement and excluded the 58 human-labelled studies. These lesion boxes and the anatomical dataset above were training assets. My classifiers did not need either annotation file, or a detector pass, at test time.

3. Anatomy-aware MRI classification

The classifier combines a shared 2D image backbone with target-specific spatial and depth aggregation. The study-level probabilities supervise its disease predictions; the anatomical and lesion assets guide where it gathers evidence during training.

Preserve image detail and physical geometry

We retained the raw DICOMs and cached each decoded series as a **uint16 volume with geometry metadata**. Preprocessing applied DICOM rescale values, resolved slice ordering from patient coordinates where available, and standardized intensity polarity. Nonzero finite intensities were clipped at the 0.5th/99.5th percentiles before quantization. The cache was therefore normalized, not lossless, but avoided committing early to JPEG, a fixed resize or a cropped ROI.

Physical-field-of-view construction and model transforms came afterward. Training and submission shared the preprocessing implementation, and cache-versus-online checks helped catch input drift before deployment.

Represent the study with five sequence roles

The input pipeline grouped sequences into sagittal, coronal, and axial fluid-sensitive fat-suppressed roles, plus sagittal and coronal structural roles. DICOM geometry supplied slice ordering and orientation. A common **140 mm field of view** preserved physical scale, with aspect-preserving crop/pad handling at **384 × 384** pixels.

Within each role, I sampled **32 slices** from the central 12–88% depth band and formed **30 overlapping three-slice windows**. Each window became a three-channel input to a shared 2D backbone. Training sampled eight windows per role; inference evaluated all thirty. A triplet contains neighboring sampled positions, which need not be consecutive source slices.

This representation let me reuse strong pretrained 2D models while retaining local depth context. Each window feature was projected to 256 dimensions and combined with learned role and window-position embeddings. Four geometry values described the window: mean normalized depth, depth span, valid-slice fraction, and the fraction of valid slices that were non-repeated. Five pooled role features formed a 1,280-dimensional representation for each target. Missing roles were masked.

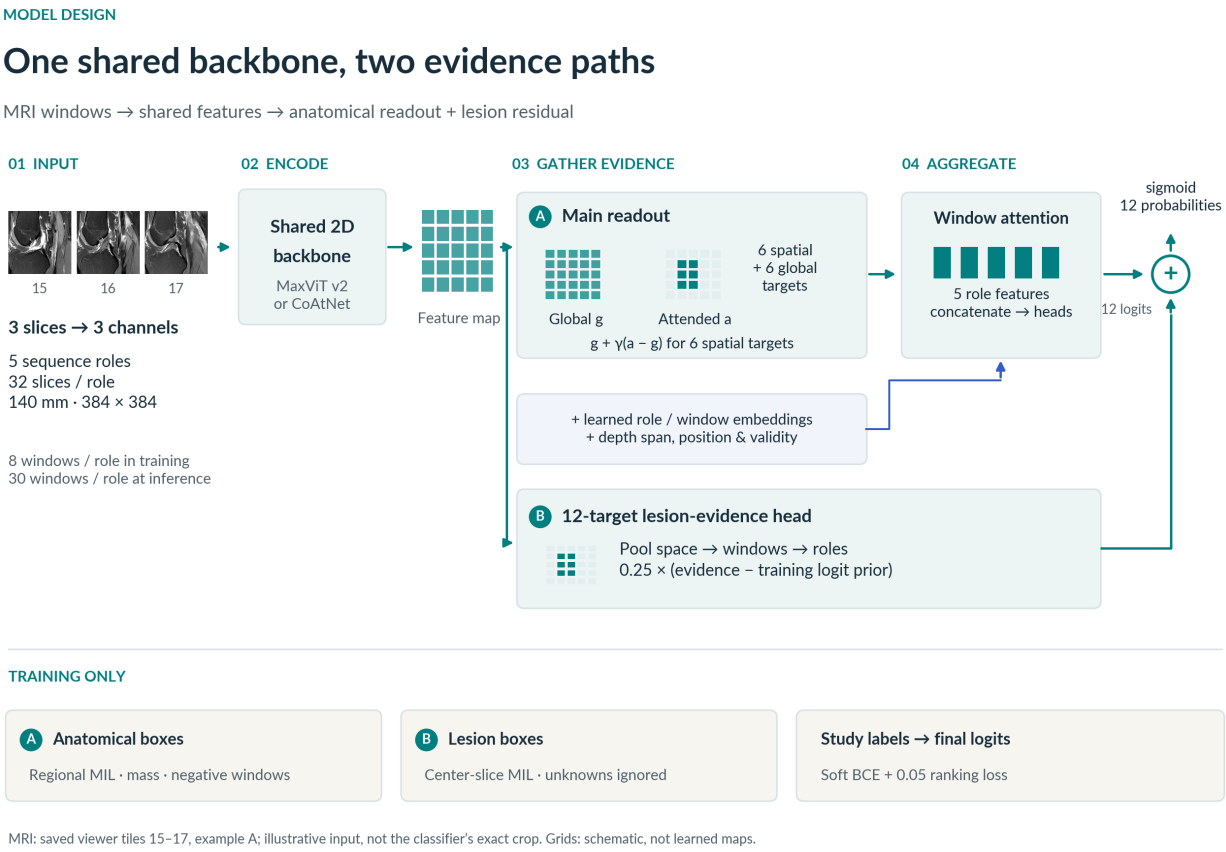


Figure 7. MRI windows share one backbone. Six targets combine global and anatomy-guided readouts; six retain global pooling. Role/window embeddings and geometry enter the study representation. A parallel lesion-evidence head contributes a prevalence-centered residual to the final logits. Spatial supervision is used only during training. Feature and attention drawings are schematic, not measured activation maps.

Preserve global context, add anatomical evidence

For six targets—MCL, medial/lateral meniscus, and medial/lateral/patellofemoral osteoarthritis—I used target-specific spatial attention. The attended feature adjusted the global average feature through a residual:

```
h_target = g + γ × (a_target − g)
```

Here `g` is global pooling, `a_target` is the spatially attended feature, and the learnable `γ` starts at 0.1. The other six targets use global spatial pooling. Each target then attends over windows within each role, and the five role features are concatenated for its classifier.

The residual starts close to global pooling and learns how strongly to shift toward localized evidence. Anatomical boxes guide it during training through regional multiple-instance learning, an attention-mass constraint, and explicitly negative-window supervision. Image and annotation transforms are synchronized.

In an earlier MaxViT v2 Base (384 px) comparison, the complete anatomy-aware package moved public AUC from **0.956 to 0.960**. The local difference was inconclusive, so I treat this as encouraging evidence for the package rather than an isolated gain from any one auxiliary loss.

I also added a **twelve-target lesion-evidence head** on the same backbone feature map. Normalized log-sum-exp pools evidence over space and valid windows, followed by an average over available roles. The final logits are:

```
z = z_main + 0.25 × (e_lesion − logit(p_train))
```

Here `p_train` is the target prevalence computed from the report-labelled training studies. Centering the evidence residual prevents simply adding another copy of the prior. Its auxiliary multiple-instance loss has weight **0.1** under the spatial-loss schedule. Unknown entries are ignored; missing annotation does not become negative supervision.

This head adds a direct route from local evidence to the study prediction without another backbone pass. It uses the train-only eligibility and geometry rules described in Section 2; the prevalence term also uses report-labelled training studies only.

4. Training and validation

The final models used released pretrained weights, soft-label BCE, a ranking loss weighted by 0.05, and the spatial auxiliary losses. Augmentation covered spatial transforms, intensity changes, depth jitter, and slice/block/role dropout. The main training settings are collected below for reproducibility.

Let the pretrained backbone adapt

A consequential training change was increasing the backbone's peak learning rate. In a matched earlier CoAtNet (224 px) comparison, moving from **8×10^{-6} to 8.944×10^{-5}** raised public AUC from **0.936 to 0.944**. The 58-study development AUC moved in the opposite direction, from 0.9283 to 0.9215. That disagreement was one reason to seek a broader development reference.

The final recipe retained the higher peak backbone LR, layer-wise decay of 0.9, and a separate 10^{-3} peak LR for new layers. The historical comparison also ran longer under its stopping rule, so it establishes the benefit of the LR intervention and resulting training trajectory, rather than isolating one internal mechanism.

Building a CV we could trust

Our initial validation on the 58 human-labelled studies was too unstable to guide development reliably. After I raised this issue, **@tascj0 led our joint development of the CV approach we used for the rest of the competition**. In our late-stage experiments, it tracked leaderboard performance much more closely and became a decisive tool for comparing label recipes and models. Establishing a useful validation signal was an important turning point: it gave us a much firmer basis for deciding which changes to pursue.

For late-stage development, we divided the report-labelled studies into **10 folds** and selected **one fold for CV based on its correlation with historical leaderboard results**. We then kept this split fixed across model comparisons. The selected fold held out **435 report-labelled studies**, with **3,914 report-labelled + 58 human-labelled studies** for training. Its main use was to compare different training-label recipes against **one fixed validation reference**: the equal blend of Astra v1, Astra v2 and teammate Astra. Changing a model's training labels did not change the ruler used to evaluate it.

We used probability-weighted AUC, or **SoftAUC**, without turning the reference probabilities into binary labels. For each target, an ordered pair of different studies `(i, j)` contributes weight `q_i × (1 − q_j)`; correctly

ordered predictions receive full credit and ties receive half. We normalized by the total pair weight and averaged over all twelve targets. This preserves uncertainty in validation as well as training, but its numerical scale is different from the competition's hard-label AUC.

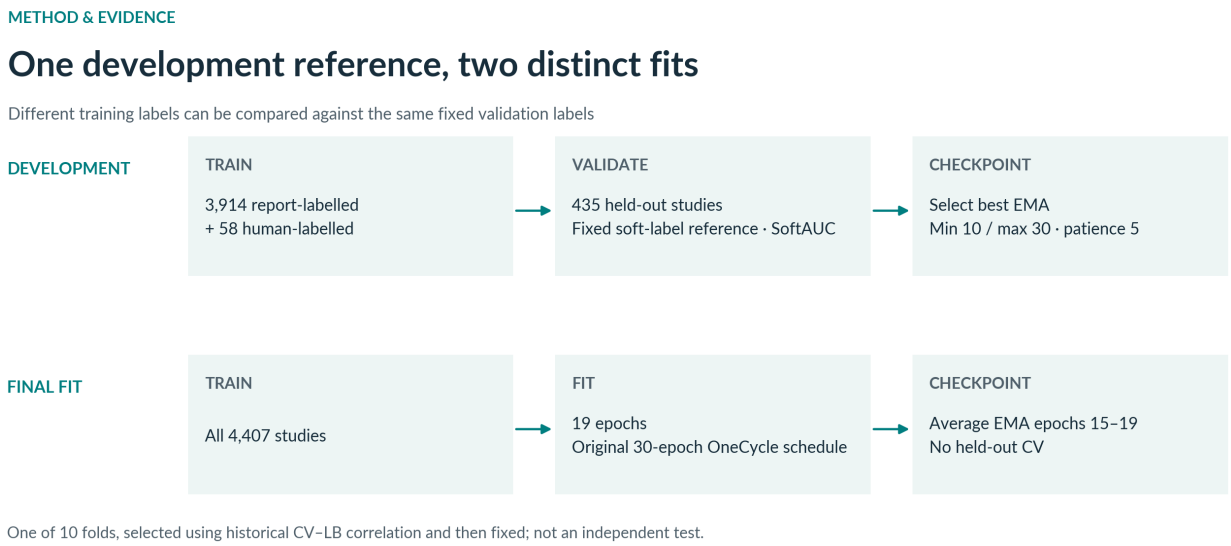


Figure 8. Development selection and final fitting use distinct data splits and checkpoint rules. Final full-data models have no held-out CV.

We selected best EMA by SoftAUC, with a minimum of ten epochs and patience five, up to thirty epochs. This was a single fixed development fold, not ten-fold averaged CV. Since its selection used historical LB results, it was not an independent test.

Full-data fitting

The final three models trained on **all 4,407 studies**, retaining the original human labels for the 58 labelled studies. Each ran for nineteen epochs on the original thirty-epoch OneCycle schedule. The deployed weights were the uniform average of **EMA epochs 15–19**, without batch-normalization recalibration.

The 58 human-labelled studies also informed annotation development. Anatomical supervision eligibility used GPT/detector agreement on their images, without using their disease labels. These choices make small local differences development evidence; validation against related annotation sources can also favor models trained on those sources.

Training recipe

Parameter	Value
Resolution / batch size	384 / 8
Optimizer / weight decay	AdamW / 0.02
Peak LR: backbone / new layers	8.944×10^{-5} / 10^{-3}
Layer-wise LR decay	0.9
Schedule / EMA decay	OneCycle, 30-epoch horizon / 0.995

Parameter	Value
Spatial auxiliary coefficients	0.1 per term, with first-epoch warmup and late decay
Training precision / memory	BF16 / activation checkpointing
Final fit / checkpoint averaging	19 epochs / EMA epochs 15–19
Packaged inference precision	FP16

The anatomy-region MIL, attention-mass, negative-window and lesion-MIL terms shared a multiplier: it increased linearly from 0 to 1 during the first epoch, stayed at 1 through epoch 10, then followed `1 − 0.7 × (e − 10) / 19`, where `e` is the zero-based epoch index. Each term had coefficient `0.1 × multiplier`. The final 19-epoch fits stopped before the 30-epoch schedule ended. The lesion-logit residual coefficient of 0.25 was separate. Checkpointed backbone blocks preserved BatchNorm running statistics during backward recomputation.

5. Results and the final contribution

How the solution developed

The work progressed through three broad stages. **First, improve what the student learns from:** the early multimodal-label CoAtNet scored 0.922 public; Astra labels brought the same student to 0.935, and the early Astra/Hulu/Qwen blend reached 0.936. **Then, improve how the student learns and reads the images:** a higher backbone learning rate reached 0.944; subsequent 384-pixel model recipes reached 0.956; the anatomical pooling and supervision package reached 0.960. **Finally, combine complementary evidence:** later labels, inputs and lesion supervision produced a 0.962 single model, before the final full-data model group.

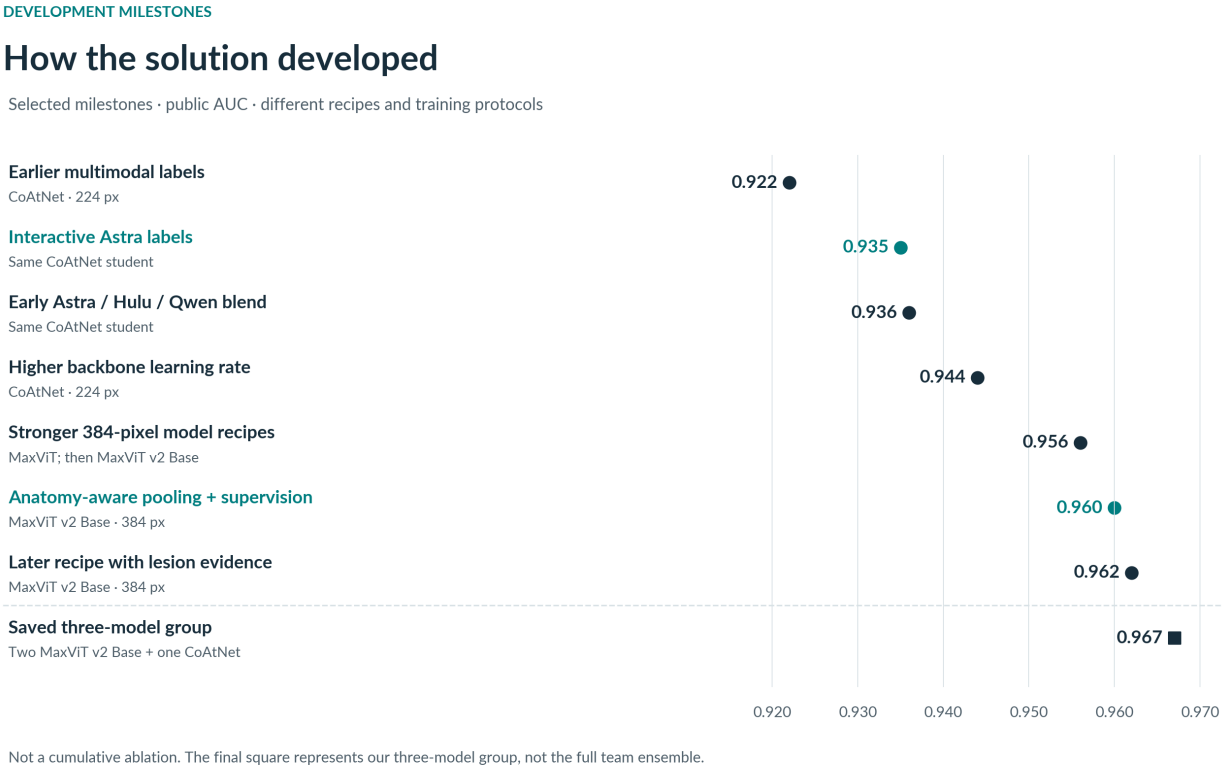


Figure 9. Selected public-LB milestones. The label comparison used the same student; the learning-rate intervention also affected training duration. Later rows include changes to model, input, labels or validation, so adjacent score differences are not isolated component gains. The final square represents our saved three-model group: 0.967 public / 0.957 private, not the complete winning team score.

The useful lesson was not that every new component helped. Early label blending added only 0.001 beyond Astra, with no confirmed advantage. The learning-rate change improved public LB while worsening the small development score. Location work began with a more expensive half-image approach before moving to shared-feature readouts. Those observations explain the final solution better than a list of its hyperparameters alone.

The final three models

My final contribution contained three models:

Backbone	Labels	Weight within my group
MaxViT v2 Base (384 px)	Mix5	1/3
MaxViT v2 Base (384 px)	Focus-guided Astra	1/3
CoAtNet-rMLP-2 (384 px)	Mix5	1/3

CV and leaderboard results

The figure below compares **five single models** for which we had both development predictions and corresponding leaderboard results.

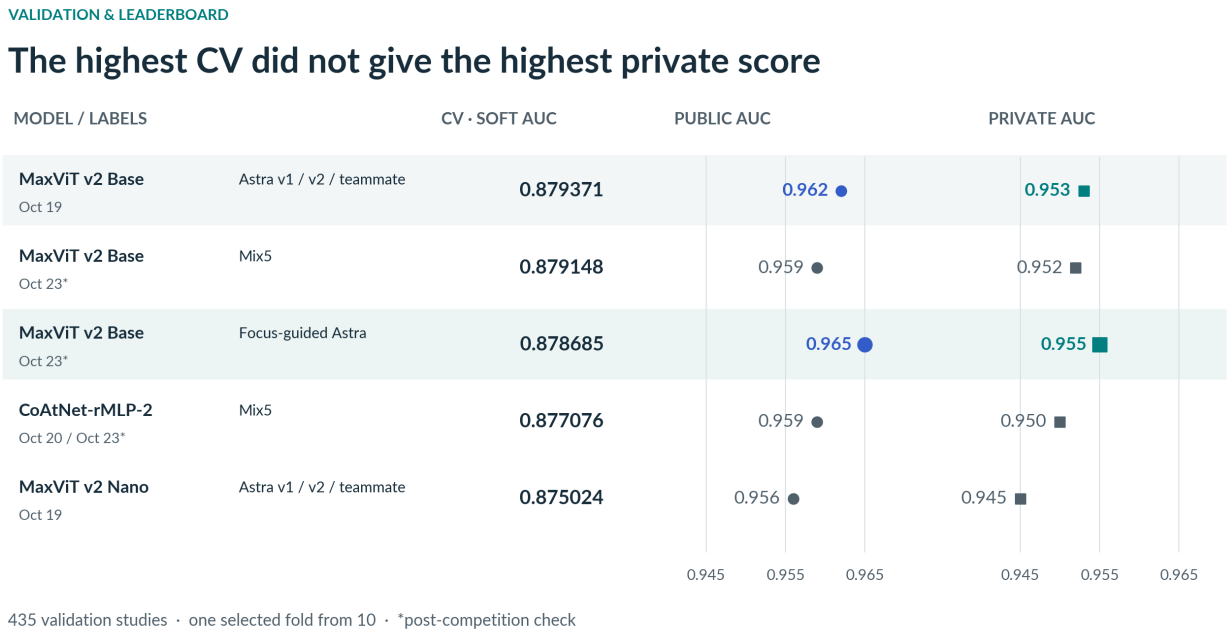


Figure 10. Each row refers to the same model version across CV and LB. MaxViT v2 Base and CoAtNet use 384-pixel inputs; MaxViT v2 Nano uses 256 pixels. The highlighted focus-guided model has lower CV than the first row but higher private AUC. The CV column is continuous-reference SoftAUC; the public and private panels show competition AUC. October 23 submissions are post-competition checks of saved models. CoAt's October 23 result repeats its earlier submission.

The MaxViT v2 Base models were competitive across both evaluations, but close CV rankings did not determine the hidden-test order. The focus-guided variant ranked below the three-source and Mix5 variants on CV, yet had the highest private AUC among the five models shown: **0.955**. These post-competition checks put small development-score differences in perspective; this small post-competition comparison is not a measurement of the broader CV–LB correlation we observed during development. The new CV was highly useful for guiding development, even though it could not resolve every close ranking.

We also evaluated complementary predictions directly. On the same 435 development studies, the equal-weight Mix5-trained MaxViT v2 Base + CoAtNet-rMLP-2 pair scored **0.880561**. Adding the focus-guided MaxViT v2 Base raised this to **0.881571**. The saved paired bootstrap interval for the difference was **[+0.000133, +0.002008]**. Several combinations were explored, so this interval is not adjusted for selection. This was a modest local contribution—below our predeclared 0.002 material-gain threshold—and is separate from the full-data models below.

Checking the three full-data models

Our three models combined **different supervision on the same backbone** with **different backbones under the same supervision**. After the competition, we evaluated the saved full-data members and their equal-weight blend separately:



Figure 11. All scores in this figure are post-competition measurements from October 23. The 0.967 public / 0.957 private result belongs to my three-model group, not the complete winning team ensemble. Full-data fits have no held-out CV.

The three-model group scored above each of its members on both public and private sets. The CV and full-data versions also differed in training coverage, duration and checkpoint averaging; their score differences cannot be assigned to additional training data alone.

Our contribution to the final team ensemble

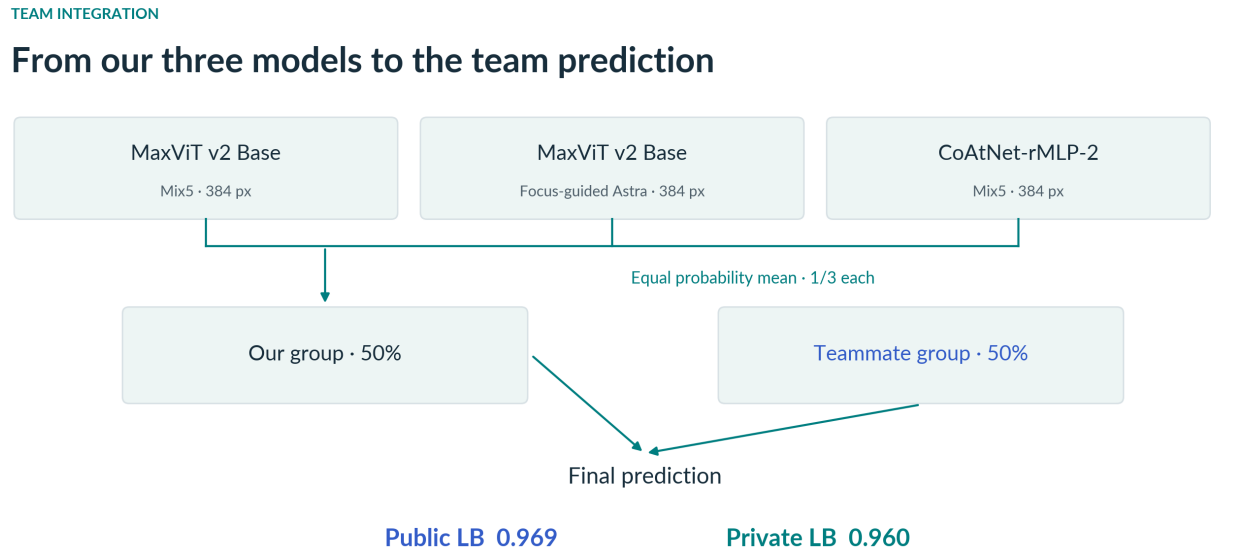


Figure 12. Equal averaging within our group, followed by equal averaging of the two team groups. The final team submission scored **0.969 public / 0.960 private**.

My three-model group contributed **50%** of the final prediction. For the other model group, please refer to @tascj0's own description. The final notebook aligned predictions by study and target and averaged the two groups, running offline on two T4 GPUs. My branch required only MRI images at inference; the annotation tools and spatial lookup tables remained training-time assets. Before integration, we bound each checkpoint to its

own input recipe and predictor, checked predictions after a cold reload, and compared packaged inference with the training implementation. These checks were important because a correct checkpoint with mismatched preprocessing is a different deployed model.

Closing thoughts

My contribution to the team connected three pieces: high-quality Astra labels, anatomical BBox annotations, and a complementary model group for the final ensemble. The most useful part of this competition for me was learning to treat annotation as something we could design, test, and improve. The report-assisted reader could inspect richer evidence than the deployed model, and the classifier could learn from that work through both continuous labels and spatial supervision.

The development lesson was to test the information being supplied: whether a label matches the competition target, whether a location is grounded in the image, and whether a student can use either. Fluent explanations and a successful auxiliary task were not enough. Matched student comparisons and spatial interventions helped decide where to invest effort; the final system brought those lessons together without carrying the annotation machinery into inference.

Thanks to RSNA, the radiologists and Kaggle for the competition, and especially to @tascj0 for the collaboration. I learned a great deal from developing the solution together, discussing ideas, and working through the challenges along the way. I hope the saved annotation example and implementation details make this part of our solution useful to others.

Training labels and spatial annotations